Mandol 超长记忆增强

系统分析与扩展设计报告

姓名：王浚沣　学号：2024K8009909018

2026 年 10 月 28 日

1　项目背景与任务说明

本项目选择开源项目 Mandol 作为研究对象，研究方向为“超长记忆增强”。本报告以源码中的 `MemoryOrchestrator` 批量构建流程作为标准改进路径，拟在一批原始记忆完成 L0 预处理和三类高层记忆构建后，增加显式的生命周期维护步骤。生命周期管理是本项目拟实现的扩展，当前 Mandol 源码尚未包含该模块。

1.1　Mandol 项目简介

Mandol 面向大模型智能体提供记忆组织、存储和检索能力。调用方将原始内容组织为记忆单元并写入语义系统；需要构建高层记忆时，可显式调用 `MemoryOrchestrator` 对一批原始记忆进行预处理并生成摘要、事实、事件、实体和关系。检索阶段按路由和配置从相应空间及索引中取得候选记忆。

1.2　研究问题

我的改进关注以下问题：

1. 长期运行后，不同会话产生的重复记忆如何识别？
2. 用户事实发生变化时，如何保留历史并判断当前版本？
3. 如何降低低价值记忆对存储、索引和上下文窗口的占用？
4. 如何在减少记忆规模的同时，保持检索质量和可追溯性？

2 Mandol 现有系统分析

2.1 总体架构

Mandol 有多种接入方式。本报告聚焦库内 `MemoryOrchestrator` 的批量处理链路；聊天示例服务只是应用示例，不作为本报告的标准流程。目标链路概括如下：

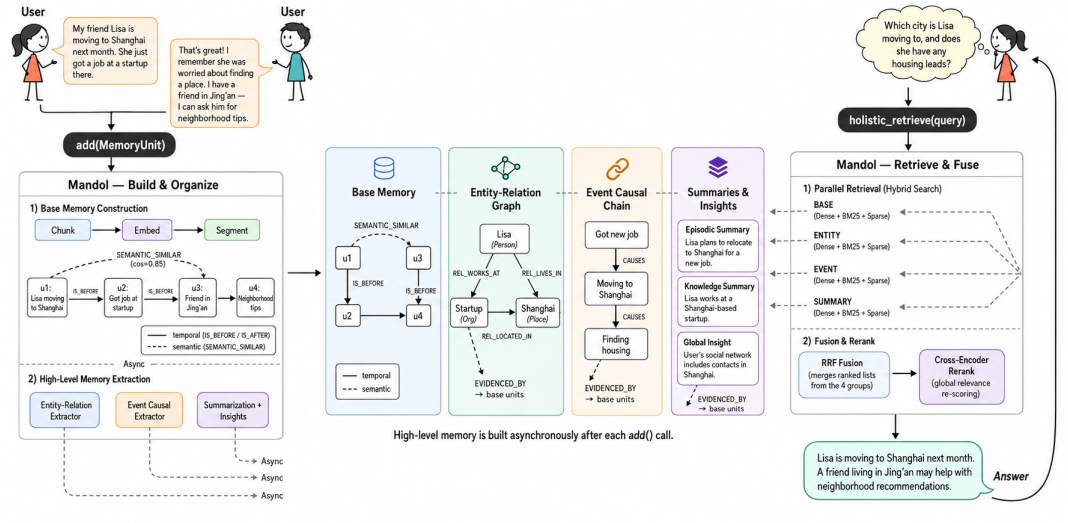


图 1: Mandol 总体架构示意

从代码调用关系看，系统可以分为以下四部分：

- 输入与调度层：调用方提供原始记忆，`MemoryOrchestrator` 负责预处理和调度构建阶段；
- 记忆对象层：使用 `MemoryUnit` 表示单条记忆；
- 存储和关系层：由 `SemanticMap` 和 `SemanticGraph` 管理内容、索引和关系；
- 构建与检索层：按配置生成高层记忆，并由检索组件从一个或多个记忆塔中找回候选内容。

2.2 MemoryUnit：统一的记忆对象

`MemoryUnit` 是系统中最基本的记忆对象，主要包含：

- `uid`：记忆的唯一标识；
- `raw_data`：原始内容；
- `metadata`：来源、层级、会话和内容类型等信息；
- `embedding`：稠密向量；

- `sparse_embedding`：稀疏向量；
- `created_time`：对象中的时间属性；当前构造函数将其初始化为 `None`，实际创建时间通常由 metadata 中的时间字段承载。

这样设计以后，不同来源的内容可以使用同一种对象表示。原始对话、L1 摘要、情景事实和实体节点都可以转换为 `MemoryUnit`，后续可以使用相同的存储和检索接口。

2.3 MemorySpace：记忆的逻辑组织

`MemorySpace` 主要保存记忆 UID 及子空间关系，不重复保存完整的记忆内容。系统使用不同空间区分不同类型的记忆。下表短名称用于说明，括号中为源码中的规范空间名；原始空间名可由 Orchestrator 配置：

空间	内容	作用
raw	原始记忆	Orchestrator 默认读取样本的空间
L0	观察单元或切片	作为高层构建器输入
L1/L2	摘要与洞察	表达更高层语义
episodic	事实与事件记忆	表达发生了什么
entity/relation	实体与关系	表达实体及其关系

表 1: Mandol 中的主要记忆空间

2.4 SemanticMap：内容、索引和访问统计

`SemanticMap` 负责保存 `MemoryUnit`，维护 `MemorySpace`，生成 embedding，更新向量索引，并记录访问次数和最近访问时间。主要功能如下：

1. 校验并保存记忆单元；
2. 将记忆加入指定空间；
3. 按配置尝试生成稠密/稀疏表示并更新索引；
4. 记录访问统计；
5. 提供删除、空间管理和存储管理能力；语义去重由部分记忆构建器在特定阶段执行，并非 SemanticMap 的通用去重接口。

当前系统已经记录访问次数和最近访问时间，这些数据可以作为后续价值评分的输入。SemanticMap 对相同 UID 的插入或更新有处理逻辑，构建器也提供局部去重；但仓库没有统一的价值评分和跨会话生命周期去重流程，无法保证不同会话中语义重复的记忆会被识别和维护。

2.5 SemanticGraph：表达记忆之间的关系

`SemanticGraph` 在 `SemanticMap` 的基础上增加图节点和图边。图节点底层对应 `MemoryUnit`，图边可以表示实体关系、来源关系、时序关系和事件关系。

因此，两个类的关系可以概括为：

`MemoryUnit`：保存单条记忆内容

↓

`SemanticMap`：管理内容、空间和索引

↓

`SemanticGraph`：表达节点之间的关系

2.6 MemoryOrchestrator 批量构建流程

本报告以 `MemoryOrchestrator.process_raw_units()` 为标准入口。调用方传入一批已创建的原始 `MemoryUnit`。Orchestrator 根据策略分配或补齐会话信息，预处理出 L0 units，再按配置执行层次、情景和实体关系构建。默认情况下，相应 builder 会将生成的记忆写回语义系统；此行为可由配置关闭。该流程是显式批处理，不表示每次单条消息写入都会自动构建高层记忆。

Listing 1: 标准批量建塔流程（生命周期维护为本项目拟增加步骤）

```
MemoryOrchestrator.process_raw_units(raw_units, ...)
    -> assign / infer sessions
    -> preprocess raw units into L0
    -> HierarchicalAutoBuilder (strategy may enable L1/L2)
    -> EpisodicAutoBuilder (if enabled)
    -> EntityRelationAutoBuilder (if enabled)
    -> builders write generated units to SemanticMap / SemanticGraph
    -> [proposed] MemoryLifecycleManager.maintain(...)
```

Orchestrator 的配置默认启用三类构建器，但层次阶段是否生成 L1/L2 还受当前 `PipelineStrategy` 控制；各阶段也可通过配置关闭。默认开启情景与实体去重，层次

去重默认关闭。这些是构建阶段内的局部去重设置，不等同于本项目拟实现的跨会话生命周期维护。

3 三种记忆塔与检索机制

3.1 三种记忆塔

Mandol 的自动构建过程由不同构建器完成。三个构建器的分工如下：

构建器	生成内容	主要回答的问题
`HierarchicalAutoBuilder`	L1 摘要、L2 总结	这段对话主要讲了什么？
`EpisodicAutoBuilder`	事实、事件、时间信息	发生了什么？什么时候发生？
`EntityRelationAutoBuilder`	实体节点、关系边	谁和谁有什么关系？

表 2: 三种记忆塔的职责

三种构建方向由 `MemoryOrchestrator` 统一调度，并共享 `MemoryUnit`、`SemanticMap` 和 `SemanticGraph`。它们是按配置运行的构建阶段，不意味着每次调用都必然执行所有阶段。

3.2 检索流程

检索阶段从多个来源取得候选结果，再进行融合、排序和裁剪。

记忆的构建和检索流程如下图所示：

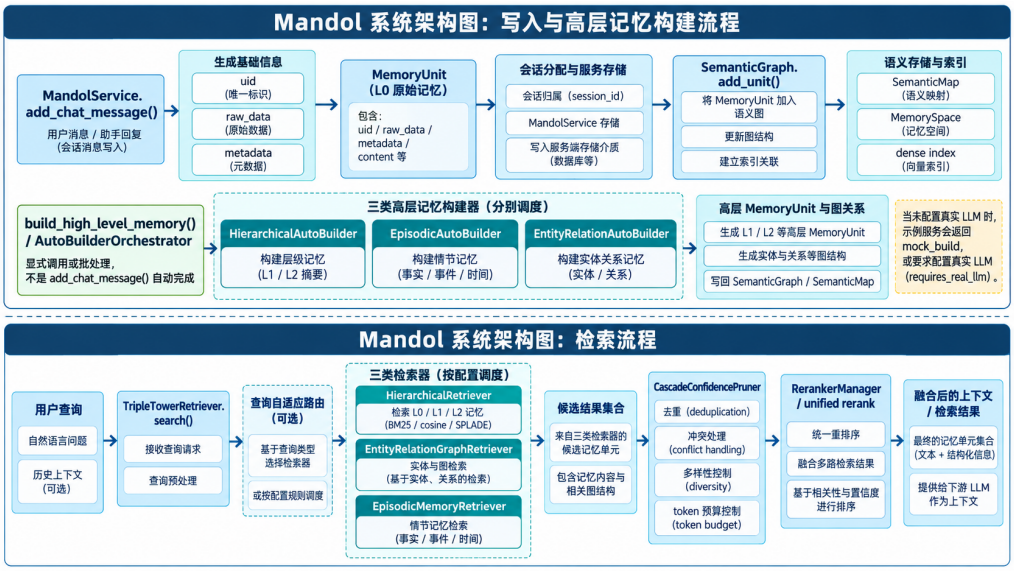


图 2: 记忆构建与检索流程

- `TripleTowerRetriever`：根据查询路由和配置调度一个或多个检索塔；
- 各塔检索器可调用底层检索方法；实际使用关键词、稠密向量、稀疏向量或图检索取决于传入方法和检索配置；
- `CascadeConfidencePruner`：处理重复、冲突、多样性和 Token 预算。

这里需要区分两种行为。检索阶段的裁剪只影响当前查询的候选上下文，不会永久删除记忆。因此，长期运行还需要单独的生命周期管理模块。

4 需求分析与问题定义

4.1 长期运行场景

随着会话数量增加，系统中可能出现以下问题：

1. 多个会话重复表达相同事实；
2. 原始对话、摘要和事实同时保留，语义重复增加；
3. 低价值闲聊长期占据索引和存储；
4. 用户信息发生变化，但旧版本仍可能被检索；
5. 记忆规模变大后，检索延迟和上下文 Token 数增加。

4.2 功能需求

针对以上问题，新模块需要实现以下功能：

- 记忆价值评估；
- 跨会话语义重复检测；
- 相似记忆合并和来源保留；
- 事实冲突识别和版本管理；
- 低价值记忆压缩或软归档；
- 更新记忆空间、图关系和向量索引；
- 输出维护过程和结果报告。

4.3 非功能需求

- 低侵入：尽量复用现有接口；
- 可追溯：合并后仍能找到原始来源；
- 可扩展：评分和合并策略可以替换；
- 可测试：每个策略可以独立单元测试；
- 可控开销：不在每条消息写入时同步调用大模型。

5 现有系统与需求的差距

新需求	当前已有能力	仍然存在的差距
语义去重	UID 去重、局部构建阶段去重	缺少长期跨会话去重
价值评估	访问次数和最近访问时间	缺少统一评分模型
事实更新	检索阶段冲突裁剪	缺少历史版本和当前版本管理
低价值处理	冷热存储	物理位置不等于内容价值
生命周期	记忆可增加、删除	缺少统一状态和状态转换规则

表 3: 现有能力与新需求的差距

6 超长记忆增强方案

6.1 总体架构

本项目拟增加 `MemoryLifecycleManager` 作为 `MemoryOrchestrator` 批量流程后的显式维护阶段，整体关系如下：

调用方提供 raw units

↓

`MemoryOrchestrator`：会话整理、L0 预处理、三类建塔并写回

↓

`MemoryLifecycleManager`（拟增加）：对选定维护范围执行评分、去重、冲突处理与策略决策

↓

同步维护决定至 `SemanticMap`、`SemanticGraph` 和索引

该模块作为 Orchestrator 批次完成后的显式阶段，不在每条消息写入时运行，也不替换三类建塔器或检索器。第一版先利用 metadata 保存生命周期字段，示例如下：

Listing 2: 拟增加的生命周期元数据

```
{
    "memory_status": "ACTIVE",
    "importance_score": 0.0,
    "last_evaluated": "...",
    "compressed_into": None,
    "source_uids": [],
    "valid_from": None,
    "valid_to": None,
    "is_current": True
}
```

6.2 价值评分

记忆价值由多个维度共同决定，初步评分公式如下：

$$V(m) = \alpha I + \beta A + \gamma R + \delta C + \epsilon F - \lambda D \tag{1}$$

其中：

- I：语义重要性；
- A：访问频率；
- R：时间新鲜度；
- C：图关系中心性；
- F：跨会话出现频率；
- D：与其他记忆的重复度。

各项权重暂不固定，后续通过对比实验确定。

6.3 语义重复检测

重复检测按照以下步骤进行：

1. 根据 embedding 查询局部近邻；
2. 过滤掉明显不同空间或不同主题的候选；
3. 计算语义相似度；
4. 结合时间、实体和来源判断是否可以合并；
5. 保存代表节点和所有源 UID。

初始实验暂定相似度不低于 0.90 时进入合并候选，0.75 至 0.90 时继续判断，低于 0.75 时暂时保留。上述阈值需要通过实验验证。

6.4 事实冲突与版本关系

例如：

$$\text{用户住在北京} \quad \longrightarrow \quad \text{用户搬到了上海}$$

两条内容不属于简单重复。处理时需要：

- 保留历史事实；
- 标记旧事实不再是当前版本；
- 标记新事实为当前版本；
- 保存有效时间；
- 在 `SemanticGraph` 中建立 `supersedes` 关系。

这样可以同时支持当前状态查询和历史状态追溯。

6.5 生命周期状态

本方案使用以下五种状态:

状态	含义
ACTIVE	正常参与检索
CANDIDATE	可能重复或价值下降，等待判断
PROTECTED	重要事实、实体或高中心性节点
COMPRESSED	已合并到代表节点或摘要节点
ARCHIVED	暂时退出主要检索，但保留恢复能力

表 4: 记忆生命周期状态

7 面向对象设计

7.1 新增类及职责

类	主要职责	代表方法
`MemoryLifecycleManager`	编排完整维护流程	`maintain()`、`generate_report()`
`ValueScorer`	计算记忆价值	`score()`、`score_batch()`
`RedundancyDetector`	查找相似记忆和重复簇	`find_similar()`、`build_clusters()`
`ConflictResolver`	识别冲突并维护版本	`detect_conflict()`、`resolve_versions()`
`MaintenancePolicy`	根据评分和关系做决策	`decide()`、`should_protect()`
`MaintenanceReport`	记录维护结果	统计合并、归档和异常

表 5: 生命周期管理模块的对象设计

7.2 对象调用关系

Listing 3: 生命周期维护的拟调用流程

```
def maintain(units):
    scores = scorer.score_batch(units)
    groups = detector.build_clusters(units)
    versions = resolver.resolve_versions(groups)
    decisions = policy.decide(scores, groups, versions)
    apply_decisions(decisions)
    return MaintenanceReport(...)
```

7.3 面向对象原则

- 封装：每个类隐藏自己的算法和内部数据；
- 单一职责：评分、检测、冲突处理和策略判断分开；
- 开闭原则：替换算法时不修改核心检索器；
- 低耦合：Orchestrator 在批次建塔完成后显式调用维护模块，通过现有 `SemanticMap` 和 `SemanticGraph` 接口读取和应用维护结果。

8 关键流程与实现计划

8.1 维护触发时机

本项目的标准接入点是 `MemoryOrchestrator` 完成一次批量构建之后。维护范围由本次样本及其相关历史记忆组成，需在实现中明确限定；其他触发方式不属于本次标准实现路径。维护不在每条消息写入时执行。Orchestrator 流程拟调整为：

- 接收一批原始记忆并完成 L0 预处理；
- 按策略执行已启用的层次、情景和实体关系构建阶段；
- 将构建结果写回语义系统；
- 显式调用 `MemoryLifecycleManager.maintain()`，输出维护报告。

8.2 完整维护流程

1. 收集新增记忆和相关历史候选；
2. 计算价值评分；
3. 查找语义近邻；

4. 判断重复或冲突；
5. 执行保护、合并、压缩或归档；
6. 更新 MemorySpace、SemanticMap 和 SemanticGraph；
7. 输出 MaintenanceReport。

8.3 代码实现分阶段

阶段	主要任务	验收标准
第一阶段	实现价值评分和统计读取	评分结果稳定，边界测试通过
第二阶段	实现 embedding 近邻和重复候选分组	能发现跨会话相似记忆
第三阶段	实现来源保留和冲突版本关系	合并后可以追溯，冲突事实不被误删
第四阶段	接入 Orchestrator 批量建塔完成后的维护调用	对本次维护范围生效，且不破坏原有检索流程
第五阶段	完成消融实验和参数调整	得到规模、质量和延迟对比结果

9 测试与实验设计

9.1 单元测试

单元测试至少覆盖以下情况：

- 空输入和单条输入；
- 完全重复的记忆；
- 高度相似但主题不同的记忆；
- 同一事实的多个版本；
- 零访问次数和极端时间戳；
- 重要实体节点的保护。

9.2 图一致性测试

完成合并、压缩和归档后，检查以下内容：

- 所有边的两端节点仍然存在；
- 来源 UID 可以继续追溯；
- 版本边的方向正确；
- MemorySpace 与 SemanticMap 的成员关系一致；
- 索引中不存在已经失效的节点。

9.3 对比实验

实验设置四组方案进行比较：

1. 原始 Mandol；
2. 只启用价值评分；
3. 价值评分加语义合并；
4. 完整生命周期方案。

9.4 评价指标

指标类别	指标
规模	记忆总数、重复率、压缩率、内存占用
检索质量	Recall@K、MRR、当前事实命中率、历史保留率
上下文	Token 数、重复上下文比例
性能	平均延迟、P95 延迟、维护耗时
可靠性	图悬空边数量、来源追溯成功率

表 7: 实验评价指标

10 总结与后续工作

Mandol 已经提供统一的记忆对象、记忆空间、向量索引、语义图、由 `MemoryOrchestrator` 调度的三类记忆构建器和多路检索机制。现有系统已有部分构建阶段去重及基于访问信

息的容量管理，但尚无统一的跨会话生命周期维护流程。

本项目拟在 MemoryOrchestrator 批量建塔完成后调用 MemoryLifecycleManager，对明确的维护范围执行价值评分、语义重复检测、冲突版本管理、生命周期状态决策和报告生成。上述生命周期能力属于本项目扩展，只有在完成实现和验证后才能作为已实现功能陈述。

A 主要源码阅读路径

- src/mandol/core/memory_unit.py
- src/mandol/core/memory_space.py
- src/mandol/core/semantic_map.py
- src/mandol/core/semantic_graph.py
- src/mandol/auto_builder/orchestrator.py
- src/mandol/auto_builder/hierarchical_builder.py
- src/mandol/auto_builder/episodic_builder.py
- src/mandol/auto_builder/entity_relation_builder.py
- src/mandol/retrieval/advance_retriever.py
- src/mandol/triple_retrieval/triple_tower_retriever.py

B 大模型使用说明

本项目开发过程中使用了 OpenAI GPT-5 系列大模型，并通过 Codex 完成代码阅读和辅助开发。大模型主要用于以下工作：

1. 辅助阅读 Mandol 源码，梳理 MemoryUnit、SemanticMap、SemanticGraph 和 MemoryOrchestrator 之间的关系；
2. 根据类定义和方法调用整理基础记忆写入、高层记忆构建以及三塔检索流程；
3. 协助分析超长运行场景下的重复记忆、事实冲突和低价值记忆问题；
4. 辅助设计 MemoryLifecycleManager、ValueScorer 和 RedundancyDetector 等扩展类；
5. 协助设计单元测试、图一致性测试和对比实验指标；

大模型生成的内容没有直接作为最终结论使用。对于类名、方法名、调用顺序和现有功能，均通过项目源码进行核对；对于改进方案，则区分已有功能和拟新增功能。实验数据、运行结果和最终设计决策由本人根据实际代码和测试结果确定。大模型仅作为代码理解、方案分析和文档整理的辅助工具。